

Figure 4: Functional description of a neuron

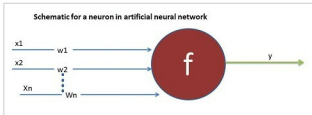


Figure 5: Schematic for a neuron in an artificial neural network

Note 1: In this article, I have tried my best to make you understand the basic concepts of deep learning, the differences between machine learning and AI, as well as some basic algorithms. The scope of this article does not allow a description of all the algorithms with mathematical functions.

Basics about neural networks

At its core, the neuron is optimised such that it receives information from other neurons and, in parallel, processes this information in a unique way, before sending the outcome (results) to other cells. This process works as shown in Figure 4.

Because of the advances mentioned earlier, humans are now able to simulate artificial neural networks using algorithms and computers. Each of the incoming connections is dynamically strengthened or weakened based on how many times it is used (this is also how humans learn new concepts). After being weighted by the strength of the respective connections, the inputs (Figure 5) are summed together in the cell body. This sum is then transferred to a new signal for other neurons to catch and analyse (this is called propagation along the cell's axon and sent off to other neurons).

Using mathematical vector forms, this can be represented as follows:

Inputs as a vector $X = [x_1, x_2 \dots x_n]$

And weights of the neuron as $W = [w_1, w_2 \dots w_n]$

Therefore the output Y can be represented as $Y = f(X \cdot W + b)$
Where "b" is the bias term.

As such, many of these bunches of individual artificial neurons are connected together to form an ANN (artificial neural network). This ANN is a combo of various input and output layers along with hidden layers (as depicted in Figure 6).

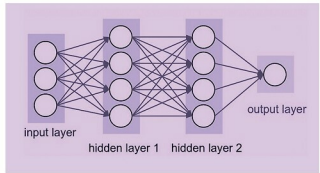


Figure 6: Artificial neural networks

Note 2: A hidden layer neuron is one whose output is connected to the inputs of other neurons and is therefore not visible as a network output (hence the term 'hidden layer').

Note 3: Refer to the article 'Do deep nets really need to be deep?' by Lei Jimmy Ba and Rich Caruana for more details.

Some time back, in 2014, it was observed that a training set with a huge data set in a shallow net, with one fully connected feed-forward hidden layer on the available data, yielded 86 per cent of the test data. But if this same data set was trained in a deeper neural net consisting of a convolutional layer, pooling layer, and three fully-connected feed-forward layers on the same data, 91 per cent accuracy on the same test set was obtained. This 5 per cent increase in accuracy of the deep net over the shallow net occurs because of the following reasons:

- The deep net has more parameters.
- The deep net can learn more complex functions, given the same number of parameters.
- The deep net has better bias and learns more interesting/ useful functions leading to invention and improvement of many algorithms.

A few algorithms

The scope of this article allows me to describe just a few important algorithms, based on these learnings and improvements. The following are the most used and popular algorithms. They are categorised as training feed-forward neural networks.

Gradient descent: Imagine there is a ball inside a bucket and the goal is to 'get as low as possible'. This requires optimisation. In this case, the ball is optimising its position (left to right or right to left, based on the condition) to find the lowest point in the bucket.

The only information available is the slope of the side of the bucket at its current position, pictured with the blue line in Figure 7. Notice that when the slope is negative (downward from left to right), the ball should move to the right. However, when the slope is positive, the ball should move to the left. As